Chapter 9. Composing the asynchronous

This chapter covers

- The limits of processing data synchronously
- How an event loop works and why you need to think asynchronously
- Events as data and data as events
- Event emitters and how to compose event streams

Remember imperative programs? They're a bit like the list of chores that Scruffy's mum used to leave him:

- Clean your room
- Take out the trash
- Do your homework
- No games until you finish your chores!

Scruffy could do the chores in any order, but he couldn't play until the chores were finished. If everything is synchronous, then games just go after chores:

```
chores()
games()
```

You've learned about things that are asynchronous. If Scruffy pays Agtron to do his chores, then he can start his games immediately. Unfortunately, if Agtron never finishes the chores, then Scruffy is in big trouble. You learned that the solution to this in CoffeeScript is to use a callback:

```
chores games
```

This is the equivalent of Scruffy asking Agtron to tell him when the chores are done so that he can start playing games. It's also the programming model used in almost all CoffeeScript programs.